Recommendation System Using the MovieLens Dataset

Topics

- User-based recommendation
- Similarity measures
- Predicting ratings
- Evaluating recommender systems
- Project: improving the recommendation algorithm

Why Recommender Systems?

- Modern platforms rely heavily on recommendation systems.
- Examples:
  - Netflix → movie recommendations
  - Amazon → product suggestions
  - Spotify → music playlists
  - YouTube → video recommendations
  - Others
- Goal:
  - Predict what a user will like

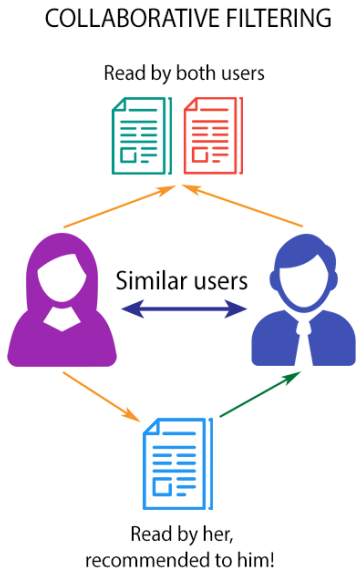
The Key Idea

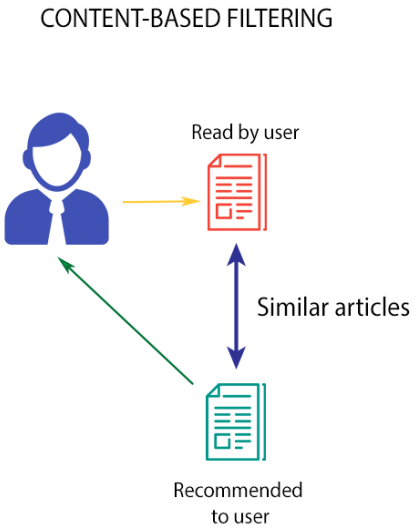
- Recommendation systems answer this question: "People similar to you liked what?"
- Example:
  - You liked:
    - Toy Story 
    - Star Wars 
  - Other users who liked those movies also liked:
    - The Matrix
    - Avatar
  - Recommend those movies to you.

Collaborative Filtering

- Collaborative filtering uses **user behavior**.
- Two major approaches:

Approach	**Idea**
User-based	Find users similar to you
Item-based	Find movies similar to movies you liked





we start with **user-based collaborative filtering**.

The MovieLens Dataset

- MovieLens 100K contains:
  - 100,000 movie ratings
  - 943 users
  - 1682 movies
- Important files:
  - u.data → ratings
  - u.item → movie information
  - u.user → user demographics
- Ratings range: 1 (dislike) → 5 (love)

Rating Matrix Representation

- The dataset can be represented as a matrix.

	Movie1	Movie2	Movie3	Movie4	Movie5	Movie6
User1	3	2	NA	5	4	4
User2	2	3	4	NA	NA	4
User3	NA	2	5	4	5	3
User4	3	NA	4	4	3	NA

- Rows → users
  Columns → movies
- Missing values → movies the user has not rated.

Data Sparsity

- Most entries are missing

	Movie1	Movie2	Movie3	Movie4
User1	3	NA	NA	5
User2	NA	3	4	NA
User3	NA	2	NA	4

- Problem:
  - Two users may share **only one common movie**
  - Similarity estimates may be unreliable

User-Based Collaborative Filtering (CF)

- Pipeline:
  - Load rating data
  - Compute similarity between users
  - Find similar users (neighbors)
  - Predict ratings using neighbors
  - Recommend highest-scoring movies

Measuring User Similarity

- We need to measure how similar two users are
- Common similarity measures:

Method	Description
Pearson correlation	measures rating pattern similarity
Cosine similarity	measures angle between rating vectors
Jaccard coefficient	used for binary preferences
Manhattan distance	distance between rating values

- In our base system we use: Pearson correlation

Why Pearson Correlation?

- Different users rate differently

Movie	Alice	Bob
Movie 1	5	3
Movie 2	4	2
Movie 3	3	1

- Alice and Bob like the **same movies**
- Bob gives lower ratings.
- Pearson adjusts for this by **centering ratings around the mean**.

Pearson Correlation Formula

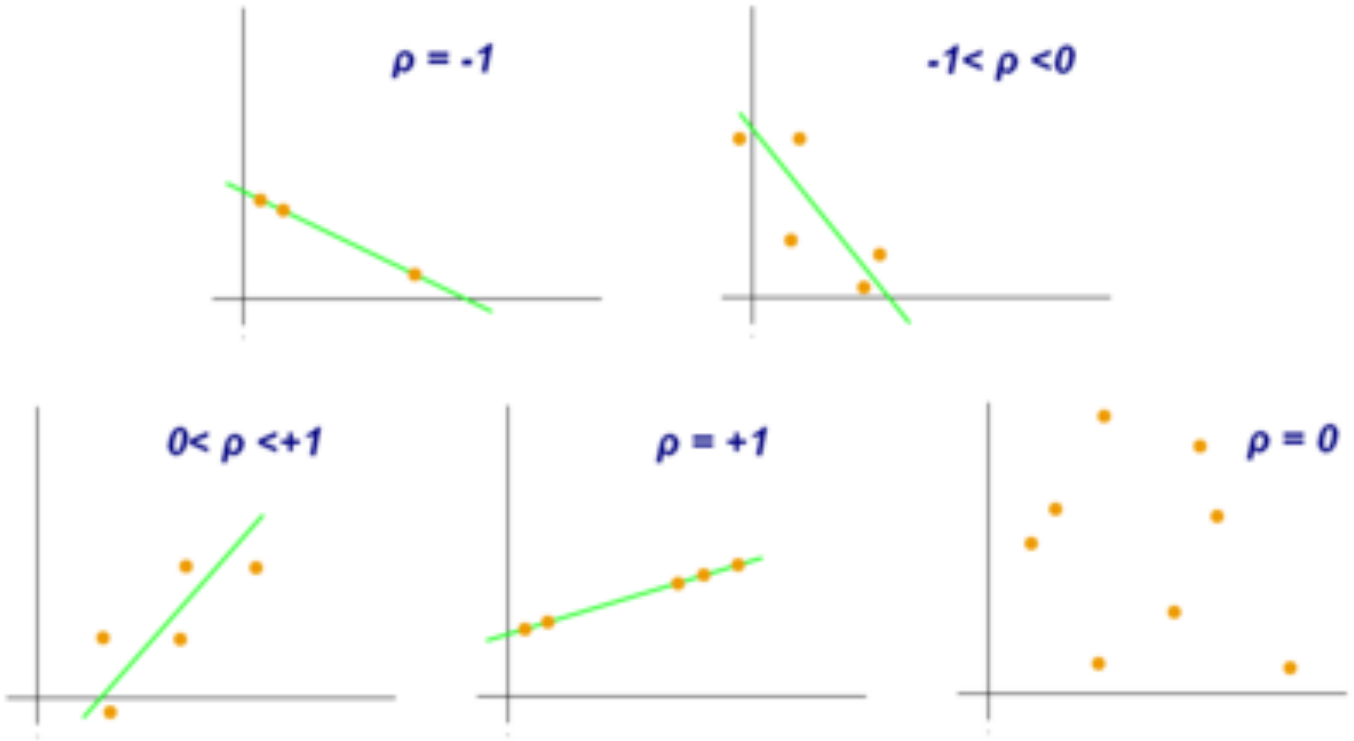
- For two users $x$and y:

$$r = \frac{\sum XY - \frac{\sum X \sum Y}{N}}{\sqrt{\left(\sum X^2 - \frac{\left(\sum X\right)^2}{N}\right)\left(\sum Y^2 - \frac{\left(\sum Y\right)^2}{N}\right)}}$$

- Interpretation:

r value	Meaning
1	perfect agreement
0	no correlation
-1	opposite preferences

Pearson Correlation



Finding Similar Users

- Example:

User	Similarity to Target
Alice	0.92
Bob	0.81
Carol	0.6
Dave	-0.2

- Typically we **ignore negative similarity users**
- Neighbors = users with positive similarity
  - Users with higher similarity influence recommendations

Predicting a Rating

- To estimate rating for: User U, Movie M
- Compute use ratings from similar users
- Prediction formula:

$$\hat{r}_{u,m} = \frac{\sum sim(u,v) \cdot r_{v,m}}{\sum |sim(u,v)|}$$

- This is a **weighted average**

Example Prediction

- Neighbor ratings:

User	Similarity	Rating	Weighted Contribution
Alice	0.9	5	0.9x5
Bob	0.7	4	0.7x4
Carol	0.3	2	0.3x2

- Prediction:

$$\hat{r} = \frac{0.9 \times 5 + 0.7 \times 4 + 0.3 \times 2}{0.9 + 0.7 + 0.3} \approx 4.16$$

From Prediction to Recommendation

- After predicting ratings for unseen movies:

Example:

Movie	Predicted Rating
Matrix	4.8
Avatar	4.5
Titanic	4.3

Recommendation:

Movies correspond to the Top-N highest predicted ratings

Limitations of Basic CF

- Problems:
  - sparse data
  - noisy similarity estimates
  - weak neighbors influence predictions
  - users with few shared movies distort results

- These issues motivate **algorithm improvements**

Possible Improvements

- We can improve recommendations by:
  - using **different similarity measures**
  - using **top-K nearest neighbors**
  - requiring **minimum shared ratings**
  - switching to **item-based recommendation**
  - using **user demographic information**
  - Deep Neural Networks for rank prediction
  - Hybrid approaches (using additional types of data)

Item-Based Collaborative Filtering

- Instead of finding **similar users**, we find **similar movies**
- Recommendation rule: If you liked Movie A, recommend movies similar to Movie A
- Example:
  - User U liked: Toy Story  and Star Wars 

Similar movies:

Movie	Similarity
Toy Story → Monsters Inc	0.92
Toy Story → Finding Nemo	0.88
Star Wars → The Empire Strikes Back	0.95

Recommended movies: Monsters Inc, Finding Nemo The Empire Strikes Back

Prediction Formula

- To estimate rating for user u on movie i:

$$\hat{r}_{u,i} = \frac{\sum_{j \in S(i)} sim(i,j) \cdot r_{u,j}}{\sum_{j \in S(i)} |sim(i,j)|}$$

Where:

- j = movies the user already rated
- sim(i,j) = similarity between movies
  - how to compute the similarity between two movies?

Item-based Similarity

- For each movie, collect ratings from all users

Example

User	Toy Story	Finding Nemo
U1	5	4
U2	4	5
U3	5	4
U4	3	2

Movie rating vectors: Toy Story = [5,4,5,3]

Finding Nemo = [4,5,4,2]

Item-Based Approach

- Advantages:
  - more stable similarity estimates
  - movies change less than user behavior
  - scales better for large systems

- Examples:
  - Amazon recommendations
  - Netflix hybrid systems

Evaluation Strategy

- How do we measure recommendation quality?

  We use **prediction accuracy**

- Approach:
  - Hold out test data
  - Predict ratings
  - Compare predicted vs actual ratings

Evaluation Procedure

- Procedure:
  - Randomly select **100 users** for testing
    random.seed(**42**)
    test_users = random.sample(all_users, 100)
  - Remaining users = training data
- Leave-One-Out Prediction Evaluation
  - For each test user :
    - hide one movie rating
    - Use the remaining ratings of the user + training data
    - predict the rating
    - compare with true rating

Measuring Prediction Quality

- For each user, compare:

Movie	Actual	Predicted
Movie1	5	4.6
Movie2	3	3.2
Movie3	2	2.4

- Compute **Pearson correlation** between predicted and actual ratings.

Final System Score

- For each test user:
  - Compute correlation value
- Final evaluation:
  - **Average** Pearson correlation across 100 users
- Higher correlation = better recommendation system

Course Project

- Goal: Improve the recommendation system
- You will implement:
  - recommendationA.py
  - recommendationB.py
- Each should modify the base algorithm

Possible Modifications

- Examples:
  - new similarity measure
  - top-K neighbors only
  - minimum overlap threshold
  - item-based recommendation
  - demographic filtering
  - Others (possibly semester project)
    - Deep Neural Networks for prediction
    - Hybrid methods
    - Use evaluation metrics such as recall at top K, Mean Average Precision, Normalized Discounted Cumulative Gain (NDCG)

Incorporating Clustering in Recommendation Systems

- **User Clustering**: Group users by rating patterns or demographics.
- **Item Clustering**: Group movies by genres or rating patterns.
- **Workflow**:
  - First, cluster users or items using algorithms like k-means.
  - Generate recommendations within or across clusters.
  - Improve cold-start users by assigning them to a cluster and recommending top items from it.
- **Benefits**: Reduces sparsity, personalizes recommendations within similar user groups, and helps overcome cold-start problems.

Deep Neural Networks in Recommendation Systems

- **Input**: User ID and Item ID (or additional features).
- **Embeddings**: Learned low-dimensional vectors for users and items.
- **Neural Network**: Captures complex, non-linear patterns in user preferences.
- **Output**: Predicted rating or relevance score.
- **Advantages**: Learns hidden factors from large data, captures complex interactions, and scales to big datasets.